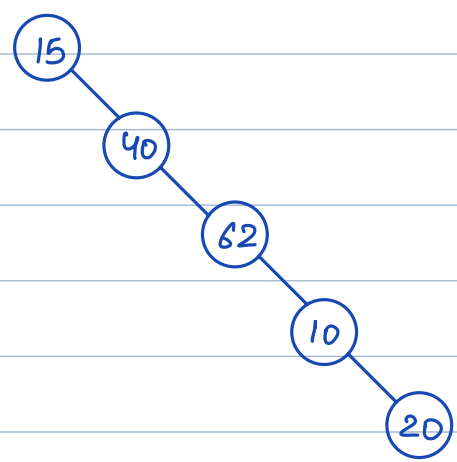
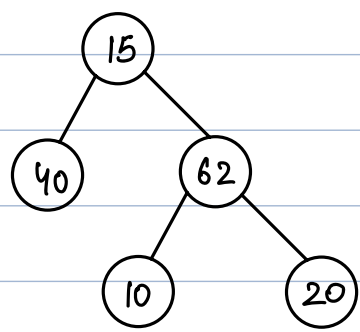
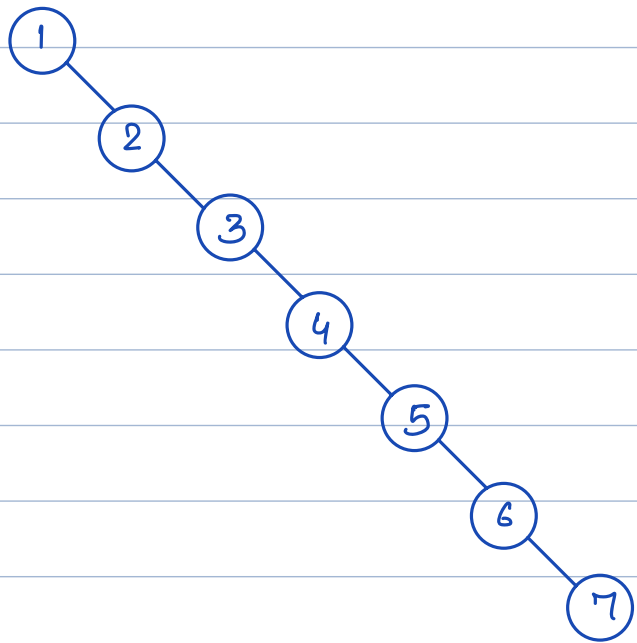
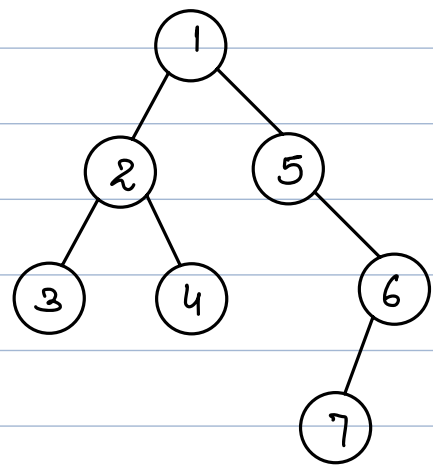
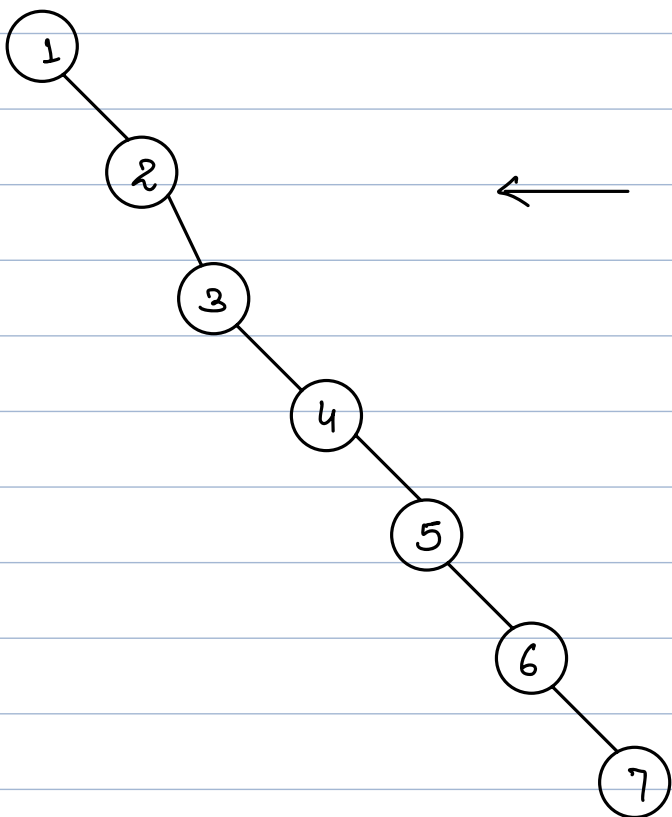
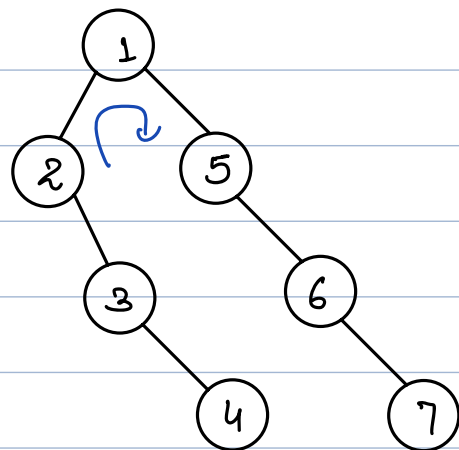
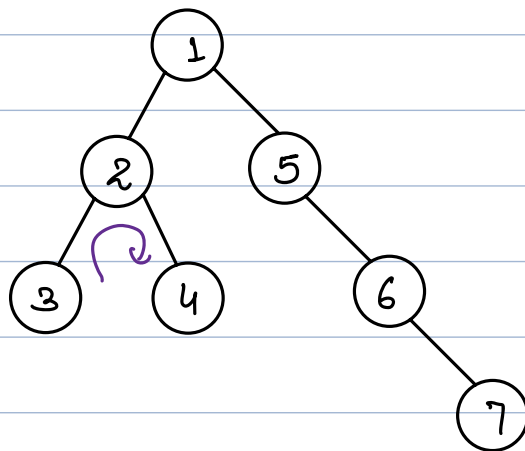
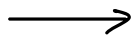
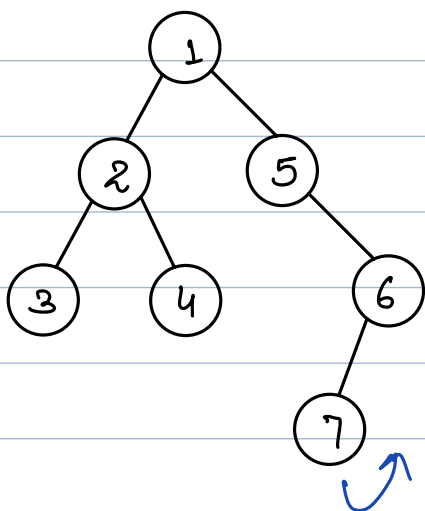


Given a Binary Tree, convert it to a Linked List where the linked list nodes follow the same order as the pre-order traversal of the binary tree. Use the right pointer of the Binary Tree as the 'next' pointer for the linked list and set the left pointer to null. Do this in place and do not create extra nodes.





```

prev = null
flatten (node) {
  if node == null
    return

```

```

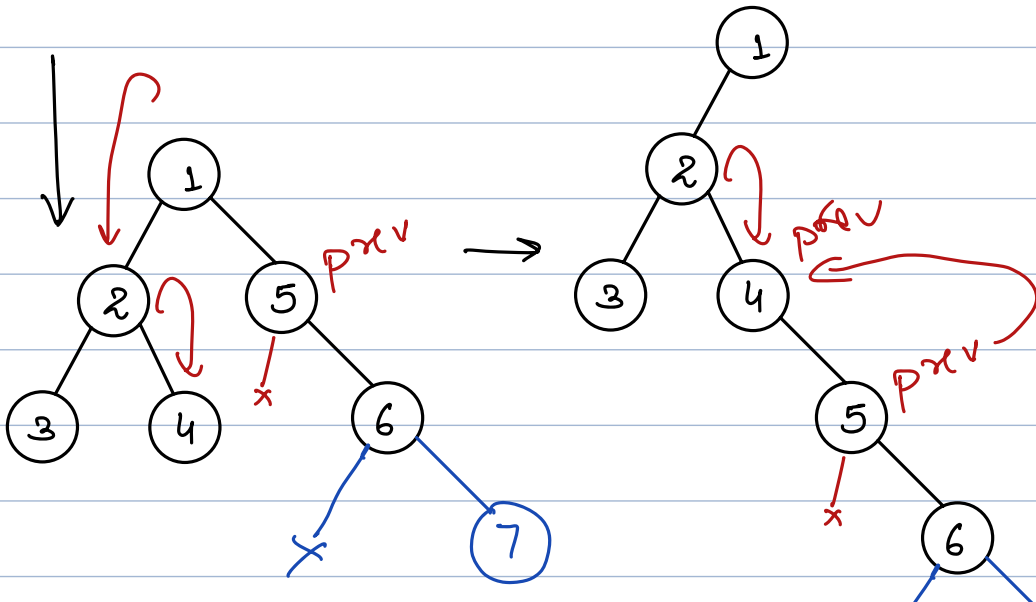
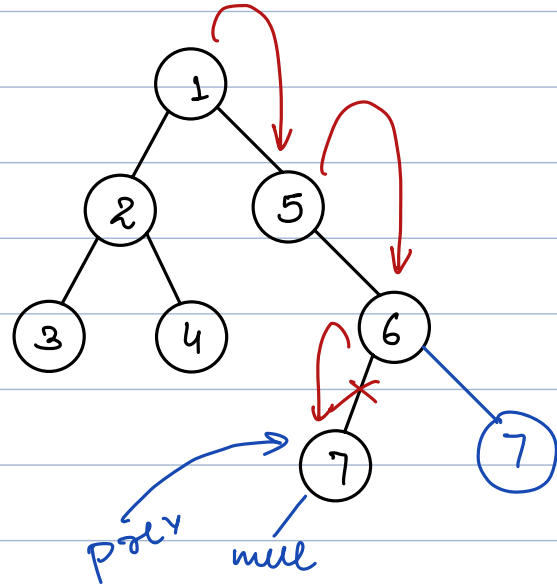
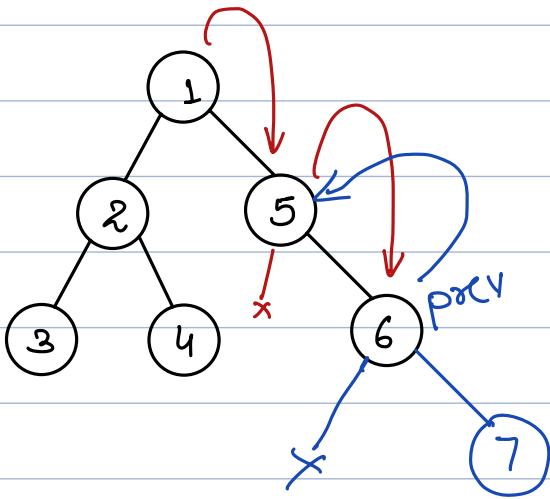
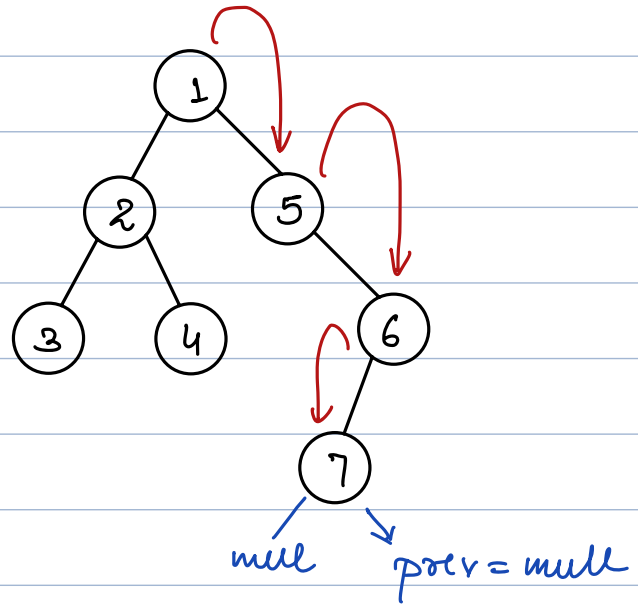
  flatten (node.right)
  flatten (node.left)

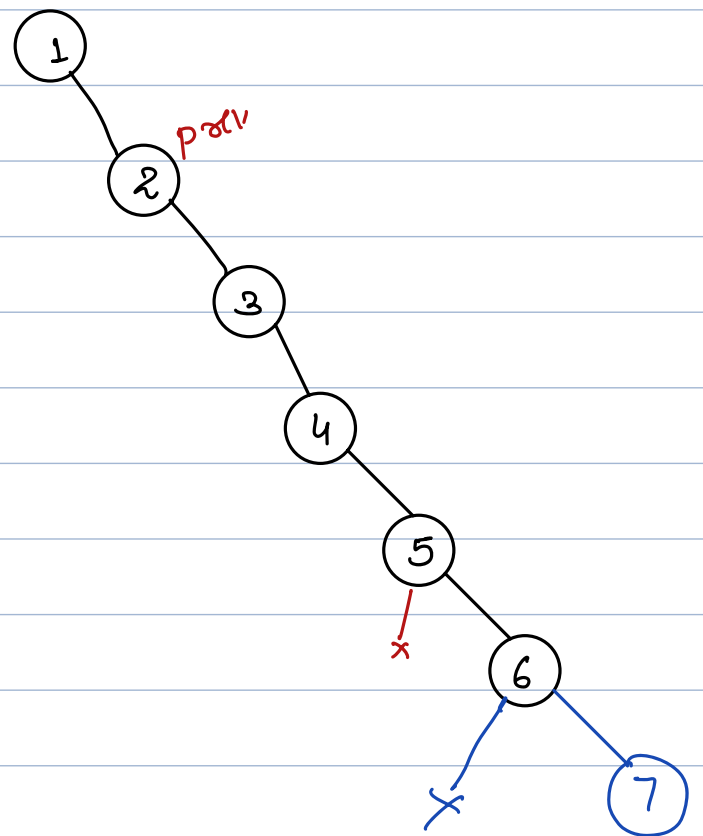
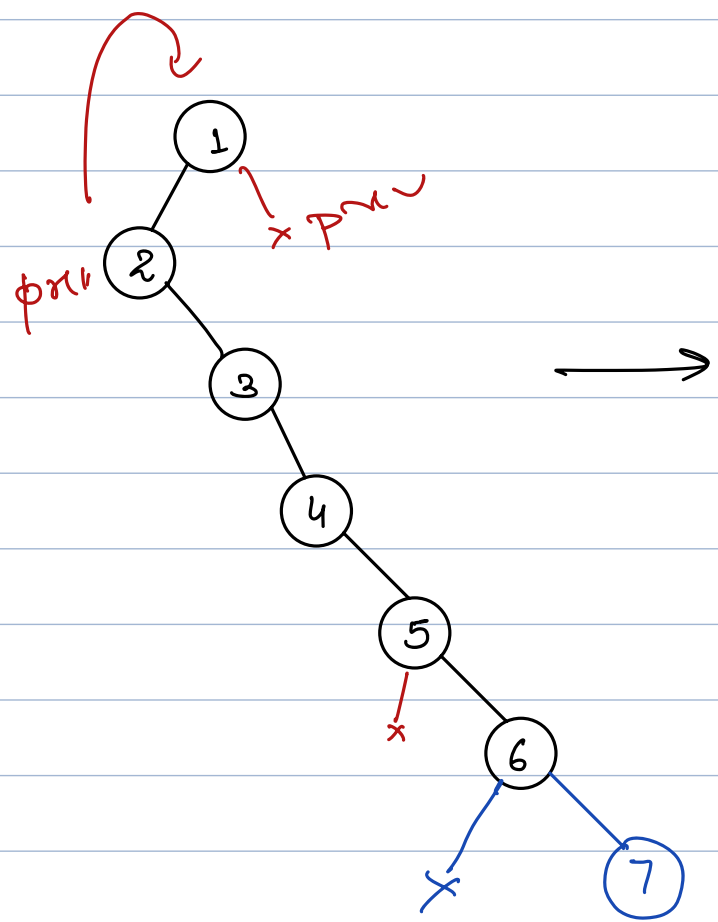
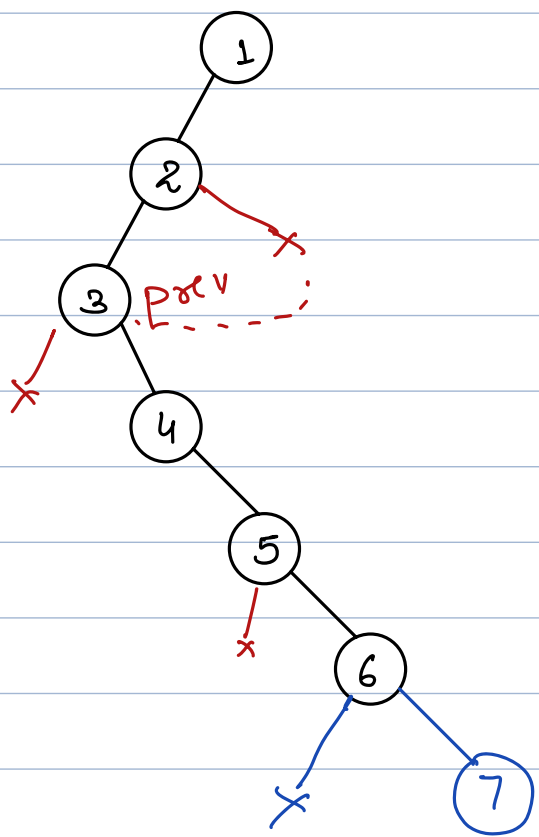
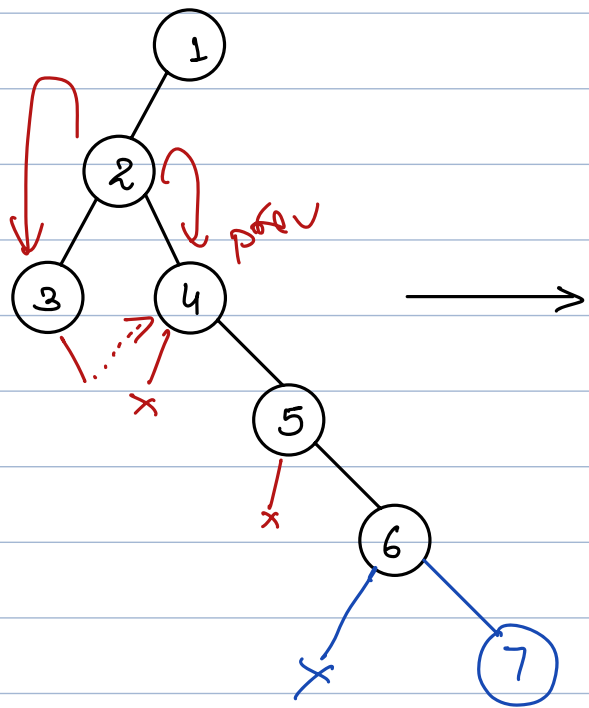
```

```

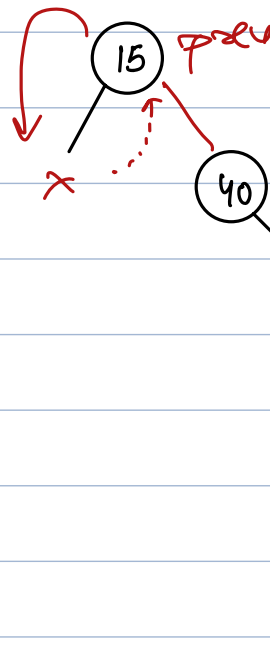
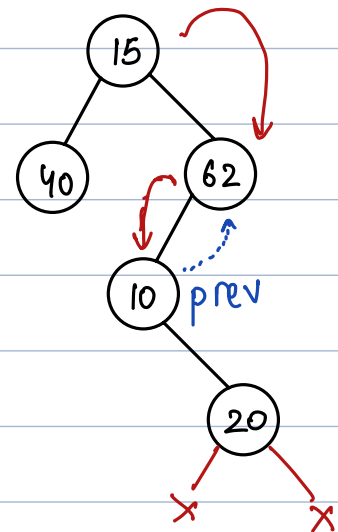
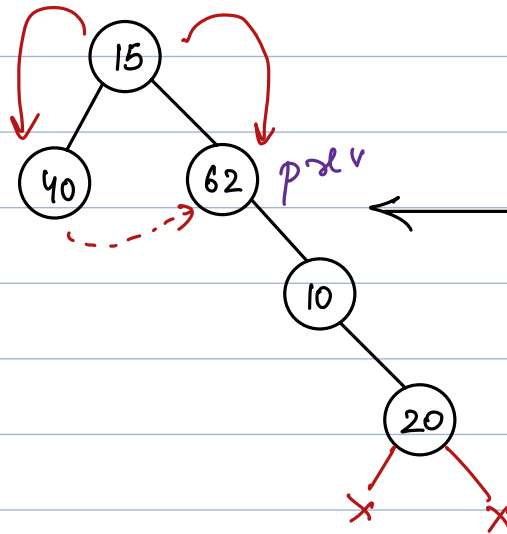
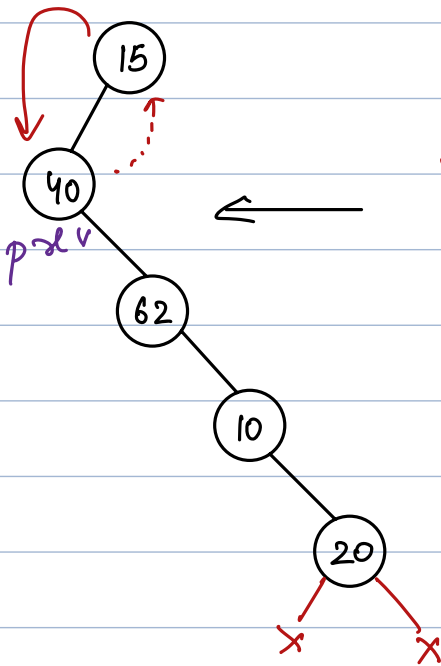
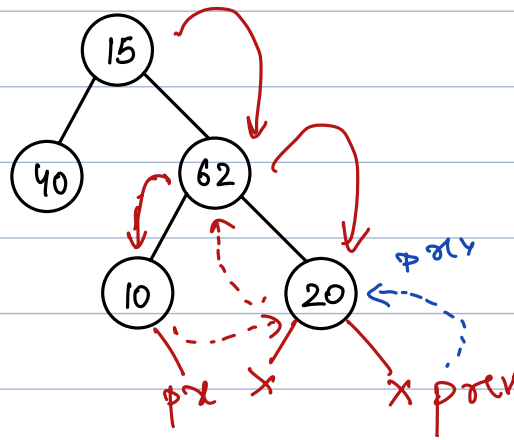
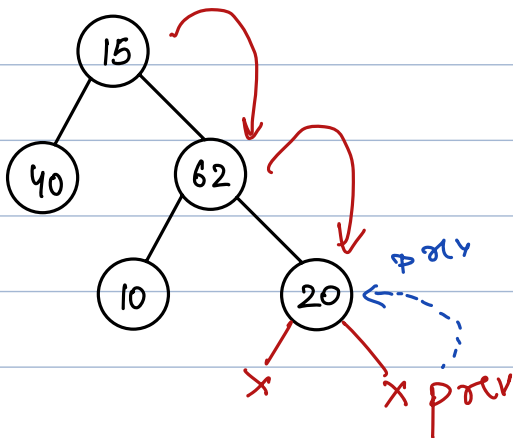
  node.right = prev
  node.left = null
  prev = node
}

```





$prev = null$



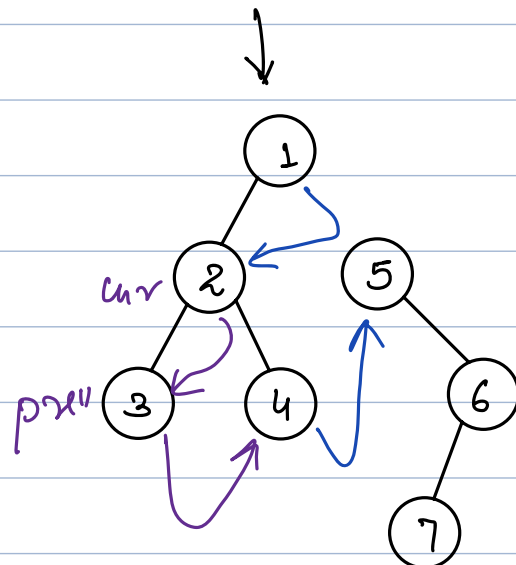
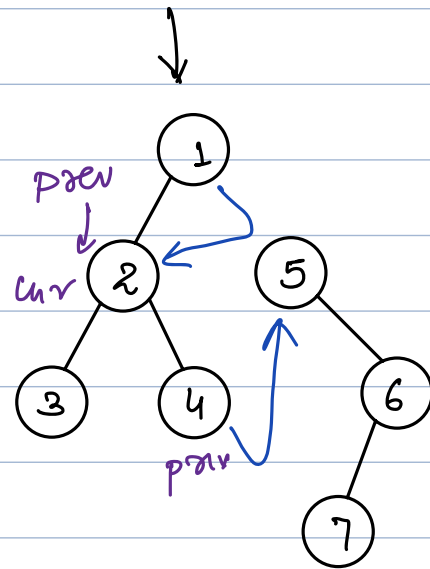
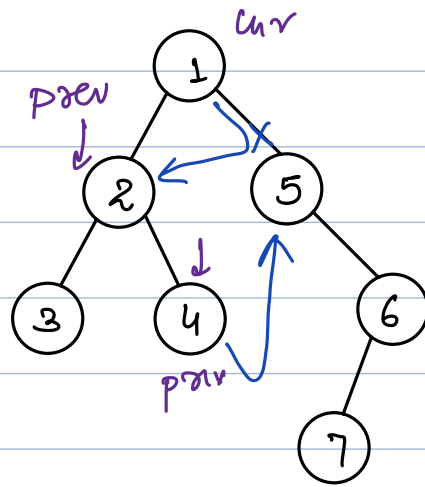
TC: $O(N)$
SC: $O(N)$

Morris Traversal

```

cur = root
while (cur != null) {
    if (cur.left != null) {
        prev = cur.left
        while (prev.right)
            prev = prev.right
        prev.right = cur
    }
    cur = cur.right
}

```



TC: $O(N)$
SC: $O(1)$

